# 1-DP-Playing with Numbers

| | |
|---|---|
| **Started on** | Saturday, 3 October 2026, 9:54 AM |
| **State** | Finished |
| **Completed on** | Saturday, 3 October 2026, 10:02 AM |
| **Time taken** | 7 mins 34 secs |
| **Grade** | **10.00** out of 10.00 (**100**%) |

**Question 1** | Correct Mark 10.00 out of 10.00 Flag question

**Playing with Numbers:**

Ram and Sita are playing with numbers by giving puzzles to each other. Now it was Ram term, so he gave Sita a positive integer 'n' and two numbers 1 and 3. He asked her to find the possible ways by which the number n can be represented using 1 and 3.Write any efficient algorithm to find the possible ways.

**Example 1:**

***Input:*** *6*

***Output:****6*

***Explanation:*** *There are 6 ways to 6 represent number with 1 and 3*

*1+1+1+1+1+1*

*3+3*

*1+1+1+3*

*1+1+3+1*

*1+3+1+1*

*3+1+1+1*

**Input Format**

First Line contains the number n

**Output Format**

**Print: The number of possible ways 'n' can be represented using 1 and 3**

Sample Input

6

Sample Output

6

**Answer:** (penalty regime: 0 %)

```c
#include <stdio.h>
long long solve(int n) {
    if (n == 0 || n == 1 || n == 2) {
        return 1;
    }
    long long dp[n + 1];
    dp[0] = 1;
    dp[1] = 1;
    dp[2] = 1;

    for (int i = 3; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 3];
    }
    return dp[n];
}

int main() {
    int n;
    if (scanf("%d", &n) == 1) {
        printf("%lld\n", solve(n));
    }
    return 0;
}
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 6 | 6 | 6 | ✔ |
| ✔ | 25 | 8641 | 8641 | ✔ |
| ✔ | 100 | 24382819506721629 | 24382819506721629 | ✔ |

Passed all tests! ✔

Correct

Marks for this submission: 10.00/10.00.

Finish review

Back to Course

# 2-DP-Playing with chessboard

| | |
|---|---|
| **Started on** | Saturday, 3 October 2026, 10:07 AM |
| **State** | Finished |
| **Completed on** | Saturday, 3 October 2026, 10:17 AM |
| **Time taken** | 9 mins 42 secs |
| **Grade** | **10.00** out of 10.00 (**100%**) |

**Question 1** Correct Mark 10.00 out of 10.00 Flag question

**Playing with Chessboard:**

Ram is given with an n*n chessboard with each cell with a monetary value. Ram stands at the (0,0), that the position of the top left white rook. He is been given a task to reach the bottom right black rook position (n-1, n-1) constrained that he needs to reach the position by traveling the maximum monetary path under the condition that he can only travel one step right or one step down the board. Help ram to achieve it by providing an efficient DP algorithm.

**Example:**

**Input**

3
1 2 4
2 3 4
8 7 1

**Output:**

19

**Explanation:**

Totally there will be 6 paths among that the optimal is
Optimal path value:1+2+8+7+1=19

**Input Format**

First Line contains the integer n
The next n lines contain the n*n chessboard values

**Output Format**

Print Maximum monetary value of the path

**Answer:** (penalty regime: 0 %)

```c
#include <stdio.h>
#define MAX_N 100

int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int grid[MAX_N][MAX_N];
    int dp[MAX_N][MAX_N];

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &grid[i][j]);
        }
    }

    dp[0][0] = grid[0][0];

    for (int j = 1; j < n; j++) {
        dp[0][j] = dp[0][j - 1] + grid[0][j];
    }

    for (int i = 1; i < n; i++) {
        dp[i][0] = dp[i - 1][0] + grid[i][0];
    }

    for (int i = 1; i < n; i++) {
        for (int j = 1; j < n; j++) {
            dp[i][j] = grid[i][j] + max(dp[i - 1][j], dp[i][j - 1]);
        }
    }

    printf("%d\n", dp[n - 1][n - 1]);
    return 0;
}
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 3<br>1 2 4<br>2 3 4<br>8 7 1 | 19 | 19 | ✔ |
| ✔ | 3<br>1 3 1<br>1 5 1<br>4 2 1 | 12 | 12 | ✔ |
| ✔ | 4<br>1 1 3 4<br>1 5 7 8<br>2 3 4 6<br>1 6 9 0 | 28 | 28 | ✔ |

Passed all tests! ✔

Correct

Marks for this submission: 10.00/10.00.

Finish review

Back to Course

# 3-DP-Longest Common Subsequence

| | |
|---|---|
| **Started on** | Saturday, 3 October 2026, 10:31 AM |
| **State** | Finished |
| **Completed on** | Saturday, 3 October 2026, 10:44 AM |
| **Time taken** | 12 mins 59 secs |
| **Marks** | 1.00/1.00 |
| **Grade** | **10.00** out of 10.00 (**100**%) |

**Question 1** | Correct Mark 1.00 out of 1.00 Flag question

Given two strings find the length of the common longest subsequence(need not be contiguous) between the two.

Example:

s1: ggtabe

s2: tgatasb

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| s1 | a | g | **g** | t | **a** | **b** | |
| s2 | **g** | x | **t** | x | **a** | y | **b** |

**The length is 4**

Solveing it using Dynamic Programming

**For example:**

| Input | Result |
|---|---|
| aab<br>azb | 2 |

**Answer:** (penalty regime: 0 %)

```c
#include <stdio.h>
#include <string.h>
#define MAX_LEN 1000

int max(int a, int b) {
    return (a > b) ? a : b;
}

int lcs(char *s1, char *s2) {
    int m = strlen(s1);
    int n = strlen(s2);

    int dp[m + 1][n + 1];

    for (int i = 0; i <= m; i++) {
        for (int j = 0; j <= n; j++) {
            if (i == 0 || j == 0) {
                dp[i][j] = 0;
            } else if (s1[i - 1] == s2[j - 1]) {
                dp[i][j] = dp[i - 1][j - 1] + 1;
            } else {
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
    }

    return dp[m][n];
}

int main() {
    char s1[MAX_LEN], s2[MAX_LEN];

    if (scanf("%s %s", s1, s2) == 2) {
        printf("%d\n", lcs(s1, s2));
    }

    return 0;
}
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | aab<br>azb | 2 | 2 | ✔ |
| ✔ | ABCD<br>ABCD | 4 | 4 | ✔ |

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Finish review

Back to Course

# 4-DP-Longest non-decreasing Subsequence

| | |
|---|---|
| **Started on** | Thursday, 8 October 2026, 9:15 AM |
| **State** | Finished |
| **Completed on** | Thursday, 8 October 2026, 9:23 AM |
| **Time taken** | 8 mins 9 secs |
| **Marks** | 1.00/1.00 |
| **Grade** | **10.00** out of 10.00 (**100**%) |

**Question 1** Correct Mark 1.00 out of 1.00 Flag question

Problem statement:

Find the length of the Longest Non-decreasing Subsequence in a given Sequence.

Eg:

Input:9

Sequence:[-1,3,4,5,2,2,2,2,3]

the subsequence is [-1,2,2,2,2,3]

Output:6

**Answer:** (penalty regime: 0 %)

```c
#include <stdio.h>

int longest_non_decreasing_subsequence(int arr[], int n) {
    if (n == 0) return 0;

    int dp[n];
    int max_len = 1;


    for (int i = 0; i < n; i++) {
        dp[i] = 1;
    }
    for (int i = 1; i < n; i++) {
        for (int j = 0; j < i; j++){
            if (arr[j] <= arr[i] && dp[j] + 1 > dp[i]) {
                dp[i] = dp[j] + 1;
            }
        }
        if (dp[i] > max_len) {
            max_len = dp[i];
        }
    }

    return max_len;
}

int main() {
    int n;
    scanf("%d",&n);
    int a[n];
    for(int i=0;i<n;i++){
        scanf("%d",&a[i]);
    }
    printf("%d\n", longest_non_decreasing_subsequence(a, n));

    return 0;
}
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 9<br>-1 3 4 5 2 2 2 2 3 | 6 | 6 | ✔ |
| ✔ | 7<br>1 2 2 4 5 7 6 | 6 | 6 | ✔ |

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Finish review

Back to Course